VOLUME II · COMPANION TO THE PROJECT GUIDE

Interview questions & answers

Why each technology was chosen, what the alternative was, and what to say when someone pushes back.

A Technology choices — 20 questions, the core of this document

B Architecture & design decisions — 8 questions

C Hard technical questions — 9 questions

D Security — 5 questions

E Pressure & curveballs — 8 questions

F Questions to ask them

G One-page cheat sheet — read this last, before you walk in

EVERY ANSWER HAS THE SAME THREE PARTS

Short answer — what you say first. Two sentences. Stop there and let them decide whether to go deeper.

If they push — the detail, including the alternative you rejected. Naming the alternative is what separates a real decision from a default.

Honest caveat — where the choice has a genuine downside. Volunteering these is a strength. Interviewers are testing whether you know the cost of your own decisions.

A note on the whole document — every "why" answer here names a real alternative. If you cannot say what you would have used instead, you did not make a choice; you accepted a default. That is fine sometimes, and three answers in here say exactly that.

PART A

Technology choices

The most likely question in any project interview, asked about any one component.

A1 Why Python and FastAPI for the backend?

SHORT ANSWER

The backend's hardest work is an AI pipeline and three API integrations, and Python has the best libraries for both. FastAPI specifically because it's async — the service spends almost all its time waiting on network calls, not computing.

IF THEY PUSH

A sync framework like Flask would block a whole worker thread on every call to Microsoft or Gmail. A sync run makes dozens of sequential HTTP requests, so with async I hold one thread and let the event loop handle the waiting.

FastAPI also gave me request validation through Pydantic for free, which I was already using for config and for validating the language model's output — so it's one validation model across the whole service rather than three.

Alternative considered: Django. Rejected because its strengths — the ORM, the admin, the batteries — are things this project either doesn't need or deliberately doesn't want. It would have been a large framework used at about 15%.

A2 Why PostgreSQL?

SHORT ANSWER

The data is deeply relational — a user has mailboxes, which have emails, which produce action items that point at contacts and at calendar events. And I rely on specific Postgres features, not just tables.

IF THEY PUSH

Four things I'd have had to build by hand elsewhere:

- **Foreign keys with cascade** — disconnecting a mailbox removes its mail automatically, in one statement, with no orphan rows possible.
- **Partial unique index** — enforces "exactly one primary mailbox per user" in the database rather than in application code that could be bypassed.
- **Advisory locks** — stop two syncs overlapping, with automatic release if the process dies.
- **Transactions** — delete-and-promote happens atomically, so a user can never end up with mailboxes but no primary.

Alternative considered: MongoDB. Rejected because the access patterns are overwhelmingly joins — "action items with their contact and their scheduled event" — which is the one thing a document store makes harder. There's no write-volume or schema-flexibility argument here that would pay for that.

A3 Why Supabase rather than plain Postgres and your own auth?

SHORT ANSWER

Almost entirely for the auth. Supabase gave me working OAuth against Microsoft and Google, and — critically — identity linking, which is what lets one user attach a second provider to an existing account.

IF THEY PUSH

Identity linking is the feature I'd least want to write myself. The flow is: a signed-in user authorises a second provider, and that provider's identity has to attach to the existing user rather than creating a new one. Getting that wrong either creates duplicate accounts or — worse — lets someone attach an identity to an account they don't own.

The database is just Postgres, so nothing about my schema or queries is tied to Supabase. If I moved off it, I'd replace the auth layer and the data would come with me unchanged.

HONEST CAVEAT

It does create one real constraint: because my `profiles` table has a foreign key into Supabase's own users table, the database and the auth service must be the same instance. That makes local development heavier than it would otherwise be — I have to run the whole stack in Docker rather than just Postgres.

A4 Why no ORM? You're writing raw SQL.

SHORT ANSWER

Because the queries that matter here are ones an ORM makes harder, not easier — upserts with conditional logic, window functions for the dashboard, and advisory locks. I'd have been writing raw SQL for those anyway, inside an ORM.

IF THEY PUSH

Three concrete examples from this codebase:

- The mailbox upsert has an `ON CONFLICT` clause where one column uses `excluded.value OR existing.value` — expressing "don't demote this account" in SQL. That's awkward in any ORM.
- The dashboard aggregates were originally a correlated subquery running once per row. Rewriting that took the query from 572 ms to 4 ms, and I verified it by reading the query planner output. An ORM puts a layer between me and that.
- `pg_try_advisory_lock` has no ORM equivalent at all.

The structure I used instead is the **repository pattern** — one class per table, holding all its SQL. So the SQL is contained and testable, and the services above it never see a query. That's the part of an ORM I actually wanted: a boundary, not an abstraction.

HONEST CAVEAT

No ORM means no compile-time protection against a typo in a column name — I catch those with tests against a real database rather than with types. On a larger team or a bigger schema I'd weigh that differently.

A5 Why Next.js, and then why export it as a static site?

SHORT ANSWER

Next.js for the routing, the build tooling and the component model. Static export because the backend already exists in Python — so there was nothing for a Next.js server to do except add a second place where data access lives.

IF THEY PUSH

If I'd used Next.js server routes, I'd have had authorisation logic in two languages and two deployments, and a second thing to keep in sync with the schema. Keeping the frontend purely a client means there's exactly one service that touches the database and holds secrets.

It also makes hosting free and effectively un-overloadable — it's files on a CDN — and it forces a clean API contract, because the frontend genuinely cannot reach around the backend.

HONEST CAVEAT

I'm using a fairly large framework for what is ultimately a client-side app, and I'm not using its headline features — no server components, no SSR, no API routes. A lighter tool like Vite with a router would have been defensible. I'd justify Next.js on the routing and build pipeline, not on the framework's main selling points, and I'd rather say that than pretend otherwise.

A6 Why TypeScript rather than JavaScript?

SHORT ANSWER

Because the frontend's main job is rendering data that arrives from an API, and almost every field can be null. Types are what stop that becoming runtime crashes.

IF THEY PUSH

Concretely: `received_at` is nullable, and in JavaScript `new Date(null)` renders the string "Invalid Date" on the page rather than throwing. Types force me to handle the null case at the point I write the component. I have a test for exactly that, because it's the kind of bug that ships.

A7 Why two languages? Isn't one simpler?

SHORT ANSWER

Each is used where it's clearly strongest: Python for the AI and integration work, TypeScript because the browser runs JavaScript regardless.

IF THEY PUSH

The honest framing is that it's one language plus the language the browser imposes. The alternative is Node for the backend to unify — but then I'm doing the AI pipeline in the ecosystem with weaker tooling for it, to avoid a context switch.

The real cost of two languages is duplicated type definitions at the boundary. I keep that cost low by having a small number of endpoints with flat response shapes.

HONEST CAVEAT

Those shared types are hand-written on both sides and could drift. Generating TypeScript types from the FastAPI OpenAPI schema would close that gap, and it's on my list.

A8 Why Tailwind for styling?

SHORT ANSWER

Speed, and no dead CSS. Styles live next to the markup they apply to, so deleting a component deletes its styles.

IF THEY PUSH

I pair it with CSS custom properties for the colour palette rather than using Tailwind's own colours. So a component says `text-[var(--color-fg)]`, and the entire theme is defined in one file. That gave me light and dark themes for free and made it possible to validate the palette for contrast in one place.

A9 Why Vitest instead of Jest?

SHORT ANSWER

It shares the project's existing build pipeline, so there's no second transform configuration to maintain, and it's substantially faster.

IF THEY PUSH

Jest historically needs its own Babel or SWC setup to understand TypeScript and JSX — a parallel build config that can drift from the real one. Vitest reuses the Vite pipeline, so what the tests compile is what the app compiles.

HONEST CAVEAT

It has sharp edges. Its fake timers break `user-event` and `waitFor` in ways Jest's don't, and I hit that — it's documented in the repo so the next person doesn't lose an hour to it.

A10 Why do you fake HTTP two different ways in the backend tests?

SHORT ANSWER

Because they test different layers. `respx` intercepts real HTTP at the transport level, and `unittest.mock` replaces a function.

IF THEY PUSH

For the client modules — the code that builds URLs, sets headers and parses responses — I use `respx`, because I want the real httpx stack to run. If I mocked the function there, I'd be testing nothing: the thing under test *is* the HTTP call.

For the sync services above them, I patch the client function and return a fixed dictionary, because there I'm testing orchestration — does it page correctly, does it delete removed items, does it store the right cursor — and the HTTP details are noise.

Choosing the fake at the right level is the point. Faking too low makes tests brittle; too high and they stop testing the thing.

A11 Why use the OpenAI SDK if you're not using OpenAI?

SHORT ANSWER

Because the provider I use is OpenAI-compatible — same request shape, same structured-output format. So the SDK is just pointed at a different base URL.

IF THEY PUSH

The practical benefit is that the provider, the model and the endpoint are all configuration, not code. Switching providers — or moving to paid OpenAI if the free quota became the limit — is an environment variable change and a redeploy.

That mattered because free-tier limits aren't a contract. They change, and a design that hard-codes one provider's quirks has to be rewritten when they do.

A12 Why a free LLM tier at all? Doesn't that cripple the project?

SHORT ANSWER

It was a requirement — I wanted this to run continuously on my own mail at zero cost. And the constraint turned out to be the most interesting engineering in the project.

IF THEY PUSH

A paid key would have let me skip the filtering, the truncation and the quota tracking entirely. Because I couldn't, I had to measure what a message actually costs, find that the budget was about forty messages a day, and get it to roughly two hundred through truncation and a pre-filter.

That's a better engineering story than "I called an API", and the mechanisms are ones you'd want at scale anyway — at a thousand users you'd care about cost per message for commercial reasons rather than quota reasons, and the work is identical.

HONEST CAVEAT

The model is a small open-weights one, so extraction quality is lower than a frontier model would give. I haven't measured accuracy systematically — I'd want a labelled set of a few hundred messages to do that properly, and that's the clearest gap in the AI side of the project.

A13 Why Fernet for encrypting the tokens?

SHORT ANSWER

It's a vetted, misuse-resistant recipe from Python's `cryptography` library. It handles the authenticated-encryption details so I can't get them wrong.

IF THEY PUSH

The failure mode I was avoiding is picking AES myself and choosing a mode without authentication — so ciphertext can be tampered with undetected — or reusing an initialisation vector. Fernet makes those decisions for me: AES-CBC with HMAC, a random IV per message, and a timestamp.

The general rule is that application code should use a recipe, not primitives.

HONEST CAVEAT

My implementation has a real weakness: there's no key version stored with the ciphertext, so the key cannot be rotated. If it leaked, every user would have to reconnect by hand. The fix is to tag each value with the key that encrypted it so old and new keys can coexist during a migration. I'd do that before this went near real users beyond me.

A14 How do you verify the user's token, and why that way?

SHORT ANSWER

PyJWT, verifying against the auth provider's published public keys rather than a shared secret.

IF THEY PUSH

With a shared secret, the same key signs and verifies — so every service that needs to verify a token could also mint one. With asymmetric keys, the auth service holds the private key and my backend only ever needs the public one, which it fetches from a well-known URL.

There's a practical reason too: newer Supabase projects default to asymmetric keys, so a shared-secret implementation simply wouldn't work. That's the kind of thing that's invisible until deployment day.

A15 Why use Microsoft's official library but hand-write the Google calls?

SHORT ANSWER

Microsoft's token flow has genuine complexity worth delegating. Google's refresh is a single form POST, and the official client library would have brought a large dependency for one request.

IF THEY PUSH

`msal` handles Microsoft's token caching and multi-tenant authority endpoints. Google's refresh is: POST four form fields, read back an access token. Writing that with `httpx` is about fifteen lines and leaves me able to see exactly what's happening.

The one Google-specific subtlety I did have to handle is that a refresh token is only issued on first consent — so if you overwrite the stored one with a null on a later refresh, you destroy the only credential that can renew the mailbox. That's a line of code, not a library.

Worth admitting: this asymmetry is a slight inconsistency in the codebase. I'd defend it on dependency weight, but "use the official SDK for both" would also have been a reasonable call.

A16 Why Render and Cloudflare Pages?

SHORT ANSWER

Both have free tiers that fit the shape of the app, and they split cleanly: a static CDN for files, a container host for the Python service.

IF THEY PUSH

Cloudflare Pages is ideal for the frontend because it's genuinely just files. Render runs a long-lived process, which the backend needs — it holds a database connection pool, so serverless functions would be a poor fit without a connection proxy.

Deployment is declared in a `render.yaml` in the repo, so the service config is version-controlled rather than clicked into a dashboard.

HONEST CAVEAT

The free tier sleeps after about fifteen minutes idle, so the first request after a pause takes tens of seconds. For a personal tool that's acceptable; for anything user-facing it isn't, and it's a paid-tier fix rather than an engineering one.

A17 Why a GitHub Actions cron instead of a proper job queue?

SHORT ANSWER

It was the smallest thing that worked, and it's free. I'd call it the weakest infrastructure decision in the project, and I know what I'd replace it with.

IF THEY PUSH

It's a scheduled workflow that sends one authenticated POST to a sync endpoint every fifteen minutes. What it doesn't give me: retries, per-user scheduling, backpressure, or visibility into what failed and why.

I did mitigate the worst failure modes in the application rather than the infrastructure. The sync holds a per-user lock so overlapping runs can't double up. One account failing doesn't stop the others. And the endpoint returns counts of synced, failed and skipped, so the run is at least observable.

With more time I'd move to a real queue — Celery with Redis, or a hosted equivalent — mainly for retries with backoff and for seeing a failing job rather than inferring it.

WHY THIS IS A GOOD ANSWER TO GIVE

Naming your own weakest decision, with the reasoning and the replacement, usually lands better than defending it. It shows you can evaluate your own work.

A18 Why Pydantic Settings for configuration?

SHORT ANSWER

So the service fails at startup if something required is missing, rather than halfway through a request at three in the morning.

IF THEY PUSH

Configuration is declared as a typed class. Required values have no default, so the process refuses to boot without them, and the error names the missing variable. Values with sensible defaults — the model name, the batch size, the truncation limit — are declared with them.

This caught a real problem: the deployment config was missing two required variables, which meant the service would have crashed on first boot instead of running in a subtly broken state. Loud beats silent.

A19 Why a Postgres advisory lock rather than Redis or a lock table?

SHORT ANSWER

Because I already have Postgres, and because an advisory lock is released automatically when the connection dies. A lock row isn't.

IF THEY PUSH

With a lock table, if the process crashes mid-sync the row stays set and that user's syncing is dead until someone clears it by hand. You then need lease timestamps and expiry logic — you're building a lock manager.

An advisory lock is held by the database session. Process dies, session ends, Postgres releases it. That's the whole failure-recovery story, for free.

Redis would work and is what I'd reach for in a distributed system, but adding a second piece of infrastructure to solve a problem the existing one already solves isn't a trade I'd make here.

One detail worth mentioning: I used try-lock rather than wait-lock. A waiting lock would queue cron workers behind one slow sync until the connection pool was exhausted. Failing fast and reporting "already running" is the correct behaviour for a scheduled job.

A20 Why httpx rather than requests?

SHORT ANSWER

It supports async, which the whole backend is. `requests` is synchronous and would block the event loop on every provider call.

IF THEY PUSH

A single sync makes dozens of sequential HTTP calls. With a blocking client, each one stalls the entire process — no other request could be served while waiting on Gmail. It also has a clean testing story via `respx`, and FastAPI's own test client is built on it, so there's one HTTP library across app and tests.

PART B

Architecture & design decisions

Less about libraries, more about why the system is shaped the way it is.

B1 Walk me through what happens when a user presses "Sync now".

SHORT ANSWER

Eight steps, and the interesting ones are the guards.

THE SEQUENCE

```
1  browser sends POST with the user's signed token
2  backend verifies the signature against published keys
3  takes a per-user advisory lock — if held, returns "already running"
4  lists that user's connected mailboxes
5  for each: decrypts its token, refreshes it if near expiry
6  dispatches on provider — Graph, Gmail, or Google Calendar
7  pages through changes, upserting rows and deleting removed ones
8  stores the new cursor, then runs extraction once for the user
```

One failing mailbox doesn't stop the others — errors are collected and the first is raised at the end, so the user with a working Outlook and an expired Gmail still gets their Outlook synced.

B2 Why is mail stored per mailbox but contacts per user?

SHORT ANSWER

Because they answer different questions. A message belongs to the inbox it arrived in. A person doesn't — they're the same person across your accounts, and unifying them is the point of the product.

IF THEY PUSH

Mail has to be per-mailbox for correctness as well as display: message IDs are only unique within a provider, so two mailboxes receiving the same ID would collide on a unique constraint. And sync bookmarks are inherently per-mailbox.

Contacts and action items being per-user is what makes "Dana emailed your work account on Monday and your personal one on Friday" a single entry rather than two. If I'd keyed them per mailbox, I'd have had to merge them in the UI on every read — doing the work repeatedly, and inconsistently.

B3 Why does one mailbox have to be "primary"?

SHORT ANSWER

Because some features have exactly one sensible target. Teams chat only exists on Microsoft, and booking a meeting has to pick one calendar to put it on.

IF THEY PUSH

Reading is different from writing. Availability checking reads *every* connected calendar — that's the whole point, so you aren't double-booked. But the new event has to be created somewhere specific, and that's the primary.

It's enforced by a partial unique index, so "exactly one primary per user" is a database guarantee rather than something application code remembers to maintain. Two bugs came out of that invariant — a token refresh silently clearing it, and deleting the primary leaving none — and both are now impossible rather than merely fixed.

B4 Why do all three providers share one code path?

SHORT ANSWER

Each provider has a client module that normalises its response into one common shape. Everything above that — the filter, the repositories, the UI — never learns which provider a message came from.

IF THEY PUSH

The clearest example is bulk-mail classification. Microsoft returns a "focused or other" field; Gmail returns category labels. Both are normalised to the same vocabulary at the client boundary, so the filter contains no provider branch at all — and it's a pure function over a dictionary, so it can be tested exhaustively without a network or a database.

The practical payoff: adding a fourth provider means a new client module and a new branch in two dispatch points. Nothing downstream changes.

B5 Why is every write an upsert?

SHORT ANSWER

Because sync is not reliably exactly-once. Making every write safe to repeat is cheaper than making sure nothing ever repeats.

IF THEY PUSH

A sync can be interrupted mid-page; a cursor can be read slightly early on purpose; a run can overlap. Rather than engineering all of that away, each write is "insert, or update if it already exists". Re-processing becomes harmless.

That's what lets me make a deliberately safe choice elsewhere — reading the new sync cursor *before* listing messages, which means mail arriving mid-sync is seen twice rather than missed entirely. Seeing it twice costs nothing because of the upsert.

THE PRINCIPLE, STATED GENERALLY

Prefer doing something twice over doing it zero times, then make doing it twice free. Duplicate work is visible and cheap; missing data is silent and permanent.

B6 How do you add a required column to a table that already has data?

SHORT ANSWER

Three steps in one migration: add it as optional, fill in the existing rows, then make it required.

IF THEY PUSH

Adding a required column directly fails immediately — every existing row violates it. The three-step version means the table is valid at every point, so if the migration fails partway the database is still consistent.

For this project the backfill filled `account_id` from each user's primary mailbox, which is correct because before the change every user had exactly one.

I verified it by replaying all fifteen migrations from an empty database and running the full suite against the result. Applying a migration to an already-migrated database proves much less — it can pass while the from-scratch path, which is the one a real deployment takes, is broken.

B7 You made some function arguments keyword-only. Why?

SHORT ANSWER

Because two of them changed meaning from "user ID" to "account ID" — both UUIDs. A stale call site still compiled, still ran, and deleted nothing while reporting success.

IF THEY PUSH

Making the parameter keyword-only turns that from a silent no-op into an immediate error. It's a small change, but it converts a class of bug from invisible to impossible.

The deeper lesson is that two values with the same type and different meanings are a design smell. The stronger fix is distinct types — a `UserId` and an `AccountId` that the type system refuses to confuse. Keyword-only is the cheap version of that idea.

B8 Why does the browser never query the database directly?

SHORT ANSWER

So there is exactly one place where authorisation happens. Two paths to the same data means two chances to get it wrong, and they drift.

IF THEY PUSH

Supabase can expose tables directly, with row-level security deciding what's visible. That's a reasonable architecture — but combined with a backend it means every rule exists twice, in SQL policies and in Python, and nothing enforces that they agree.

Because I committed to one path, I could then revoke all table permissions from the public database roles entirely. That turned "protected by policies nobody has written yet" into "unreachable by construction".

PART C

Hard technical questions

Where a senior engineer will probe. Each of these has a concrete answer from the code.

C1 What happens if a provider's API is down or rate-limiting you?

SHORT ANSWER

It depends on the status code, because they mean genuinely different things and deserve different responses.

IF THEY PUSH

STATUS	RESPONSE
`429`	Read `Retry-After`, wait, retry once. The header can be a number or an HTTP date, and both are handled.
`401`	The token expired mid-run. Refresh it and retry that one operation.
`403`	A permission that was never granted. Mark the resource unavailable and stop asking — retrying every fifteen minutes forever helps nobody.
`404 / 410`	The sync cursor expired. Fall back to a full re-sync automatically.
`5xx`	Let it fail. The next scheduled run picks up from the same cursor, because the cursor only advances on success.

The last one matters: the cursor is only stored after a page is processed successfully, so a crashed run resumes rather than skips.

C2 How do you stop the same message being processed twice by the AI?

SHORT ANSWER

Three layers: a per-user lock so runs can't overlap, a timestamp marking a message processed, and an attempt counter so failures don't retry forever.

IF THEY PUSH

The extraction query selects messages where the processed timestamp is null and the attempt count is under three, oldest first. So a successfully processed message is never selected again, and a permanently failing one drops out after three tries instead of blocking the queue.

That last part is the **poison pill** problem. Without the cap, one message that can never be parsed is retried on every run forever, consuming budget and starving everything behind it — and because the oldest are selected first, it sits at the head of the queue permanently.

Filtered-out messages are also stamped as processed, with the reason recorded. If they weren't, they'd be re-evaluated on every single run.

C3 How do you test code that depends on external APIs?

SHORT ANSWER

Three layers, with the fake placed at a different level in each.

IF THEY PUSH

- **Client modules** — `respx` intercepts at the transport layer, so the real HTTP stack runs. I'm testing URL construction and response parsing, which is the actual subject.
- **Services** — patch the client function and return a fixed dictionary. I'm testing orchestration: paging, deletion handling, cursor storage.
- **Repositories** — no faking at all. These run against a real Postgres, because the thing being tested is SQL.

The suite needs no network and runs in about forty seconds.

HONEST CAVEAT — SAY THIS BEFORE THEY FIND IT

All of it tests against fakes *I* wrote, which tests my reading of the API documentation rather than the API. The Microsoft path has run against real accounts; the Google path has not. The right fix is contract tests — record one real response per endpoint and replay it, so a change in the provider's actual shape fails the build.

C4 How do you know your tests actually test anything?

SHORT ANSWER

I write them first and watch them fail for the expected reason. A test that has never failed hasn't been shown to detect anything.

IF THEY PUSH — TWO REAL EXAMPLES

A timezone test passed when it should have failed, because my machine's offset happened to make both branches agree. I rewrote it to derive a divergent instant regardless of the machine's timezone. It would otherwise have sat there asserting nothing, forever, looking green.

When I made the test suite run against a local database instead of a remote one, I verified it by stopping the database container and re-running. It failed in exactly the same 132 places as before. That proved the tests were genuinely reaching the database rather than passing for some other reason.

C5 What's your error handling philosophy?

SHORT ANSWER

Fail loudly for programming errors, degrade gracefully for external ones, and never let one user's failure affect another's.

IF THEY PUSH

- **Isolate by unit.** One mailbox failing doesn't stop the others; one user failing doesn't stop the cron. Errors are collected and surfaced at the end rather than aborting the loop.
- **Don't let secondary work fail primary work.** If extraction fails, the sync that already fetched the messages still counts as a success — but the failure is logged with a stack trace, so it can't break silently on every run for everyone.
- **Distinguish "can't yet" from "can't ever".** A 429 is temporary; a 403 is permanent until the user acts. They get different handling.
- **Make silent failure impossible where it matters.** That's why those delete parameters became keyword-only.

C6 How do you handle timezones?

SHORT ANSWER

Everything is converted to UTC at the boundary and stored as timestamptz. Local time only appears at render time, in the browser.

IF THEY PUSH

Each provider is different: Microsoft returns times with sub-second precision Python can't parse directly and sometimes without a zone marker; Google returns the organiser's local offset. Both are normalised in the client module, so nothing downstream deals with it.

HONEST CAVEAT

All-day events are treated as UTC midnight on both providers, which is wrong outside UTC — an all-day event can block the wrong slice of a day. It's documented as a known limitation rather than fixed. The correct handling needs the event's own timezone, which both APIs do provide.

C7 Tell me about a performance problem you solved.

SHORT ANSWER

A dashboard query took 572 ms. It ran a subquery once per row. Rewriting it and adding indexes took it to 4 ms — about 141 times faster.

IF THEY PUSH

I found it by reading the query plan, not by guessing. The plan showed the inner node executing 1,000 times — once per outer row. After the rewrite it executed once.

Quoting the loop count rather than just the timing is the point: timing varies with machine load, but "1,000 loops became 1" describes the actual change in what the database does.

C8 How would you debug a user reporting "my mail isn't syncing"?

SHORT ANSWER

Narrow it by layer, because the system records state at each one.

THE ORDER I'D GO IN

1. Is the mailbox connected, and what's its status? A needs-reconnecting account explains it immediately, and the status is per account.
2. What does the sync state row say for that mailbox — is the resource marked unavailable? That means a permissions problem, not a sync problem.
3. Is there a cursor stored at all? No cursor means the first sync never completed.
4. Did the cron actually run, and what counts did it return — synced, failed, or skipped?
5. Only then look at logs for that account ID.

The design choice that makes this possible is that status and sync state are per-mailbox. With one flag per user, step one would tell me almost nothing.

C9 What would you change to support a thousand users?

SHORT ANSWER

Three things, in order: the LLM quota model, the cron, and the sync's sequentiality.

IF THEY PUSH

- **Quota.** It's tracked per provider, because the free tier's limit is organisation-wide — so one user exhausting it exhausts it for everyone. Correct for a single-user tool, unacceptable for a product. Needs per-user budgets and a paid key.
- **The cron.** One request that syncs every account in sequence. At a thousand users it wouldn't finish within its own fifteen-minute interval. Needs a queue with workers.
- **Sync concurrency.** Accounts within a user are synced one after another. They're independent, so they could run concurrently — bounded, to respect provider rate limits.

What wouldn't need to change: the schema, the per-mailbox locking, and the upsert semantics. Those were designed with concurrency in mind already.

PART D

Security

This project handles credentials to people's email. Expect scrutiny here.

D1 What's the most sensitive data here, and how is it protected?

SHORT ANSWER

The OAuth refresh tokens — more sensitive than the mail itself, because a token keeps working and grants access to everything, including mail I haven't synced yet.

IF THEY PUSH

They're encrypted with Fernet before being written, so the database stores ciphertext and the key lives in an environment variable. A database dump alone doesn't yield anyone's mailbox.

They're also never sent to the browser. The frontend has no idea these tokens exist — it holds only its own session token, which identifies the user to my backend and nothing else.

D2 How do you stop one user reading another's mail?

SHORT ANSWER

Every query is scoped by the user ID taken from the verified token — never from anything the client sends.

IF THEY PUSH

The user ID comes out of a signed token the backend verifies, so it can't be forged. Any identifier that *does* come from the client — a mailbox filter, an action item ID — is matched against the user ID as well, so a guessed ID returns nothing rather than someone else's data.

That returns "not found" rather than "forbidden", deliberately. "Forbidden" confirms the ID is real, which leaks information.

And as a second layer, the public database roles have no table permissions at all, so there's no path to the data that bypasses the backend.

D3 Can an email attack your system through the AI?

SHORT ANSWER

That's prompt injection, and it's handled by fencing: message bodies are clearly delimited in the prompt so the model treats them as content, not instructions.

IF THEY PUSH

The attack is an email containing "ignore your previous instructions and...". The risk here is bounded by design, which is worth saying: the model's output is validated against a strict schema and only ever becomes rows in two tables. It can't trigger an action, send a message, or execute anything.

The realistic worst case is a junk action item — annoying, not dangerous. If the model's output ever drove actions, fencing alone would not be enough.

D4 You found a security issue in your own project. Talk me through it.

SHORT ANSWER

No live vulnerability, but the protection was accidental rather than deliberate — which is its own kind of problem.

IF THEY PUSH

Supabase can expose tables directly over HTTP to two public roles, and by default those roles are granted full permissions on everything. Row-level security was on for every table, and eight of nine had no policies — which denies everything. So it was safe.

But it was safe because nobody had written a policy yet. Adding one policy to any table — a completely reasonable future change — would have instantly published it, and nothing about that change would have looked dangerous in review. The table holding encrypted mailbox tokens was one line of SQL from being readable.

Since the browser only does login and never reads tables, I revoked those permissions outright and set the default so new tables don't acquire them. Then I verified it: created a table and confirmed it arrived with no permissions.

WHY THIS ANSWER WORKS

It shows you can distinguish "not currently exploitable" from "safe", which is most of what security review actually is.

D5 What's the biggest security weakness you'd fix first?

SHORT ANSWER

The encryption key can't be rotated. If it leaked, there's no migration path — every user would have to reconnect every mailbox by hand.

IF THEY PUSH

The fix is versioned keys: store a key identifier alongside each ciphertext, so the system can decrypt with the old key and re-encrypt with the new one gradually. Without that, rotation is all-or-nothing, and all-or-nothing means in practice it never happens.

I'd put it above the other known gaps because it's the one where the consequence is unrecoverable rather than merely inconvenient.

PART E

Pressure questions and curveballs

Designed to see how you respond to challenge. The goal is to engage with the point, not defend reflexively.

E1 "Isn't this just CRUD with an API call in the middle?"

SHORT ANSWER

"The CRUD part was a couple of days. What took the time was everything that isn't CRUD — and I'd say the AI call is the least interesting twenty lines in the project."

THEN GIVE THREE SPECIFICS

- Incremental sync against three APIs with three different cursor models, three different ways of reporting a deletion, and three different expiry behaviours.
- Fitting a real inbox into 200,000 tokens a day — which required measuring, not guessing, and got the budget from ~40 messages to ~200.
- Changing the ownership model from one-account-per-user to many, across every table, where most of the ways to get it wrong are silent.

Don't get defensive. Agreeing that the CRUD is simple makes the rest more credible.

E2 "Why not just use Superhuman / Shortwave / an existing tool?"

SHORT ANSWER

"For the product, you probably should. I built it because I wanted the engineering problem, and because the specific thing I wanted — unified across personal and work accounts, extracting commitments rather than summarising — wasn't what those do."

IF THEY PUSH

It's also a fair question to turn into a strength: the cross-account unification is the genuinely differentiated bit. Most tools treat accounts as separate worlds. Treating a person as one person across your mailboxes is a schema decision made early and deliberately.

E3 "Is this running in production? Do people use it?"

SHORT ANSWER

Be precise, and don't overclaim. "The Microsoft path works end to end. The Google path is written and fully tested, but only against fakes — it hasn't run against a live Google account yet."

WHY THIS IS THE RIGHT ANSWER

Overclaiming collapses under one follow-up: "what happened the first time you connected Gmail?" Stating the boundary precisely reads as someone who knows the difference between tested and proven — which is exactly the judgement the question is probing.

Follow it with what you'd do: "the first thing I'd do with more time is contract tests against recorded real responses, so that gap closes permanently rather than once."

E4 "Did you use AI to write this?"

SHORT ANSWER

Answer honestly and specifically. Evasiveness is the only wrong answer, and it's increasingly a normal question rather than a trap.

WHAT MAKES AN ANSWER LAND

Be concrete about where it helped and where your judgement was the deciding factor. The things that are unmistakably yours: deciding tokens-per-day was the binding constraint and measuring it; choosing to revoke database permissions after seeing that safety came from an absence; deciding not to batch model calls because the headroom didn't justify ambiguous failures.

The real test follows anyway: they'll ask why something works the way it does. If you can explain the trade-off and the alternative — which this document exists to prepare you for — how the code was typed matters far less than whether you own the decisions.

E5 "What's the weakest part of this project?"

SHORT ANSWER

Have a real answer ready. "I can't think of one" is the worst possible response.

PICK ONE AND COMMIT

Strongest option: **the scheduling infrastructure.** A cron hitting an HTTP endpoint with no retries, no backpressure and no visibility. I mitigated the worst of it in the application — locking, per-account isolation, returned counts — but that's working around the design rather than fixing it.

Equally good: **no measurement of extraction quality.** I have no labelled set, so I can't say what fraction of action items are correct. For an AI feature that's a real gap, and I know how I'd close it.

E6 "How did you decide what to build and what to skip?"

SHORT ANSWER

By asking which half of a feature carried the value, and building that half first.

GIVE THE CLEAREST EXAMPLE

Google Calendar. *Reading* it means availability covers all your calendars, so you don't get double-booked — that's most of the value. *Writing* to it means the meeting lands on a different calendar than it otherwise would — much less value, and a second implementation of event creation.

So I built reading and documented writing as a known gap. The same reasoning kept batching out: measurably not needed yet, and it makes failures ambiguous.

The principle I'd state: prefer the half that removes a failure mode over the half that adds a capability.

E7 "What did you learn?"

SHORT ANSWER

The most reusable lesson: the dangerous bugs don't throw. They return success.

IF THEY PUSH

Nearly every real bug in this project failed silently — a token refresh that returned fine while using the wrong provider, a delete that reported success while matching zero rows, a filter rule that existed in the code and ran zero times.

A crash tells you where to look. These looked like success. That changed how I write tests: I assert that the effect happened, not that the call didn't raise. And it changed how I write functions — if passing the wrong thing can silently do nothing, I make it impossible to pass the wrong thing.

E8 "If you started over, what would you do differently?"

SHORT ANSWER

Design for multiple accounts from day one. That was the single largest source of rework.

IF THEY PUSH

The original schema made a second mailbox impossible by construction — the token table's primary key *was* the user ID. Retrofitting meant adding an account reference to every synced table, changing what uniqueness means, and re-keying the sync bookmarks, in one migration that had to be safe against existing data.

The honest nuance: starting with one account was a reasonable call at the time — it's the simplest thing that works, and I didn't know I'd want more. What I'd actually change is smaller and more general: don't make the user ID a primary key on something that is conceptually "a thing the user has", because that forecloses having two of them. That's a schema instinct I didn't have before and do now.

PART F

Questions to ask them

You'll be asked if you have any. Having none reads as disinterest; these come naturally out of what you built.

F1 On engineering practice

- "How do you handle database migrations against production data? I did nullable-then-backfill-then-not-null on this project and I'm curious whether that scales to your data sizes."
- "What does your testing split look like between unit, integration and end-to-end? I ended up with three different layers of faking and I'm not sure I got the balance right."
- "When something fails in production at 3am, what's the first thing you look at?"

F2 On how decisions get made

- "How does a technical decision get made here — is there a written proposal, or is it more informal?"
- "What's something the team chose that turned out to be wrong, and how did you find out?"
- "How much of your time goes to new features versus paying down things that are already there?"

F3 If they work with LLMs

- "How do you evaluate output quality? That's the clearest gap in my own project — I have no labelled set, so I can't say how accurate the extraction actually is."
- "Do you treat prompts as code — reviewed, versioned, tested?"
- "How do you handle cost control as volume grows?"

> WHY THESE WORK
>
> Each one references something you actually did and admits a limit of your own experience. That turns a question into a short conversation between engineers, which is far more memorable than asking about the tech stack — something you could have read on their careers page.

PART G

One-page cheat sheet

Last thing to read before you walk in. One line of justification per choice.

CHOICE	THE ONE-LINE REASON
FastAPI	Async — the backend is almost entirely waiting on network calls, not computing.
PostgreSQL	Relational data, plus I use foreign keys, partial unique indexes, transactions and advisory locks.
Supabase	For OAuth and identity linking. The database underneath is just Postgres.
No ORM	The queries that matter — conditional upserts, window functions, advisory locks — are ones an ORM makes harder.
Repository pattern	The boundary an ORM gives you, without the abstraction. One class per table.
Next.js static export	The Python backend already exists; a Next server would duplicate data access in a second language.
TypeScript	Almost every API field is nullable; types force the null case to be handled.
Tailwind + CSS vars	No dead CSS, and the whole palette lives in one file so themes and contrast are checkable.
Vitest	Reuses the app's build pipeline, so tests compile what the app compiles.
respx + mock	Fake at the transport layer for client code, at the function for services. Different layers, different tools.
OpenAI SDK → Groq	Groq is OpenAI-compatible, so provider and model are configuration rather than code.
Fernet	A vetted recipe, not primitives — so I can't pick an unauthenticated mode or reuse an IV.
JWT via JWKS	My backend only ever needs the public key; it can verify without being able to mint.
msal / raw httpx	Microsoft's token flow is genuinely complex; Google's refresh is one form POST.
Advisory lock	Released automatically when the connection dies. A lock row would stay stuck.
Pydantic Settings	Missing config crashes at startup naming the variable, not mid-request later.
Render + Pages	Backend needs a long-lived process for the connection pool; frontend is genuinely just files.
GitHub cron	Smallest thing that worked — and the weakest decision in the project. I'd replace it with a queue.

The five numbers to have ready

NUMBER	WHAT IT IS
`200,000`	Tokens per day on the free tier — the limit that binds first, ahead of requests per day.
`~40 → ~200`	Messages per day before and after truncation and filtering.
`572ms → 4ms`	The dashboard query, after rewriting a per-row subquery and adding indexes. ~141×.
`1,000 → 1`	Query planner loop count for that same fix — the number that describes what actually changed.
`381 / 186`	Backend and frontend tests. The backend suite needs no network at all.

THREE HABITS TO SHOW, IF THE CONVERSATION ALLOWS

Measure before optimising. The token budget, the query plan, the truncation ratio — all measured, which is why they're quotable.

Verify the negative. Stopping the database to confirm the tests really used it. Watching a test fail before making it pass. Creating a table to confirm it had no permissions.

Make bad states impossible, not merely absent. Partial unique indexes, keyword-only parameters, transactional delete-and-promote. Each converts a bug from "fixed" to "unrepresentable".

Companion to Volume I, the project guide. That volume explains what the system does and how it works; this one is about defending the decisions. If you only have time to reread one page before the interview, make it this one.